

qt designer를 이용해서 pyside6를 쓸 수 있어?

Qt Designer를 이용해 PySide6 UI를 제작하고 파이썬 코드와 연동할 수 있습니다. PySide6를 설치하면 Qt Designer와 UI 변환 도구가 기본 패키지에 함께 포함되어 제공됩니다.

풍경소리(정용만) - 티스토리

1. Qt Designer 실행하기

ds31x - 티스토리

PySide6 패키지를 설치한 후 터미널(또는 명령 프롬프트)에서 바로 실행할 수 있습니다.

Bash



```
pip install PySide6
pyside6-designer
```

위 명령어를 실행하면 Qt Designer 창이 열리며, 드래그 앤 드롭으로 버튼·레이아웃 등을 배치한 뒤 `.ui` 파일(예: `main.ui`)로 저장하면 됩니다.

2. PySide6에서 .ui 파일을 활용하는 2가지 방식

방법 A: `.py` 파일로 컴파일하여 사용 (실무 권장)

`pyside6-uic` 도구를 사용해 `.ui` 파일을 파이썬 코드로 변환합니다. IDE에서 코드 자동 완성을 지원하므로 가장 널리 쓰입니다.

풍경소리(정용만) - 티스토리

Bash



```
pyside6-uic main.ui -o ui_main.py
```

변환된 클래스를 파이썬 메인 스크립트에서 상속받아 호출합니다.

Python



```
import sys
from PySide6.QtWidgets import QApplication, QMainWindow
from ui_main import Ui_MainWindow # 컴파일된 UI 클래스 임포트

class MainWindow(QMainWindow, Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self) # UI 컴포넌트 초기화

        # 버튼 이벤트 연결 예시 (버튼 objectName이 pushButton일 때)
        self.pushButton.clicked.connect(self.handle_click)

    def handle_click(self):
        print("버튼이 클릭되었습니다.")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```

방법 B: QUiLoader 로 동적 로드

파일을 파이썬 코드로 변환하지 않고, 런타임에 `.ui` 파일을 직접 읽어옵니다. UI 수정 시 매번 변환 명령어를 실행할 필요가 없는 장점이 있습니다.

Python



```
import sys
from PySide6.QtWidgets import QApplication
from PySide6.QtUiTools import QUiLoader
from PySide6.QtCore import QFile

app = QApplication(sys.argv)

ui_file = QFile("main.ui")
ui_file.open(QFile.ReadOnly)

loader = QUiLoader()
window = loader.load(ui_file)
ui_file.close()
```

```
# 위젯 참조 및 이벤트 연결 예시
# window.pushButton.clicked.connect(...)

window.show()
sys.exit(app.exec())
```

- **PyQt와의 차이점:** PyQt5/PyQt6에서는 `pyuic5`, `pyuic6` 명령어를 사용하지만, PySide6에서는 `pyside6-uic` 및 `pyside6-designer` 를 사용합니다.

세슘이라는 라이브러리랑 연동할 수 있어?

PySide6 환경에서 Cesium(세슘)을 연동할 수 있습니다.

Cesium은 웹 기반의 3D 지구/공간 시각화 라이브러리(**CesiumJS**)가 표준이므로, PySide6의 내장 브라우저 엔진인 `QtWebEngineWidgets` 를 통해 데스크톱 GUI 내부에 임베딩하고 `QWebChannel` 로 Python-JavaScript 간 양방향 통신을 구성하는 방식이 가장 정석적인 연동 기법입니다.

1. 핵심 연동 아키텍처

- **렌더링:** PySide6의 `QWebEngineView` 위젯에 CesiumJS HTML 파일을 로드하여 Chromium 기반 WebGL 3D 엔진으로 렌더링.
- **데이터 및 제어 연동:** `QWebChannel` 을 이용해 Python 객체를 브라우저 JavaScript 환경에 노출하여, Python의 계산 데이터(좌표, 궤도, 상태 벡터 등)를 전달하거나 버튼 이벤트로 카메라를 제어.

2. Qt Designer 설정

Qt Designer에서 `WebEngineView`를 직접 배치하려면 일반 위젯을 승격(Promote)하여 사용합니다.

1. Qt Designer에서 기본 `QWidget` 을 폼에 배치.
2. 해당 위젯 우클릭 후 `Promote to...` (승격...) 선택.
3. 설정값 입력:
 1. **Base class name:** `QWidget`
 2. **Promoted class name:** `QWebEngineView`
 3. **Header file:** `PySide6.QtWebEngineWidgets`

4. 'Add' 후 'Promote'를 누르고 저장 (`main.ui`).

3. 기본 구현 코드 예제

필요 패키지:

Bash

```
pip install PySide6 PySide6-WebEngine
```

HTML/JS (`cesium_view.html`):

CesiumJS 리소스와 `qwebchannel.js` 를 연결하여 Python 신호를 수신하는 파일입니다.

HTML

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <script src="https://cesium.com/downloads/cesiumjs/releases/1.115/Build/Cesium/Cesium.js"></script>
  <link href="https://cesium.com/downloads/cesiumjs/releases/1.115/Build/Cesium/Widgets/widgets.css" rel="stylesheet">
  <script src="qrc:///qtwebchannel/qwebchannel.js"></script>
  <style>
    html, body, #cesiumContainer { width: 100%; height: 100%; margin: 0; padding: 0; }
  </style>
</head>
<body>
  <div id="cesiumContainer"></div>
  <script>
    // Cesium ion 기본 토큰 설정 필요 시 여기에 추가
    const viewer = new Cesium.Viewer('cesiumContainer', {
      baseLayerPicker: false,
      geocoder: false
    });

    // Python과 통신 채널 연결
    new QWebChannel(qt.webChannelTransport, function (channel) {
      window.pyBridge = channel.objects.pyBridge;

      // Python에서 전달된 함수 호출 수신
      pyBridge.sigFlyTo.connect(function (lat, lon, height) {
        viewer.camera.flyTo({
```

```

        destination: Cesium.Cartesian3.fromDegrees(lon, lat, height)
    });
});
});
</script>
</body>
</html>

```

Python 메인 코드 (main.py):

QtWebEngine을 초기화하고 Python-JS 간 브리지를 등록합니다.

Python



```

import sys
import os
from PySide6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QPushButton, QW
from PySide6.QtWebEngineWidgets import QWebEngineView
from PySide6.QtWebChannel import QWebChannel
from PySide6.QtCore import QObject, Signal, Slot, QUrl

# Python -> JS 통신용 브리지 객체
class Bridge(QObject):
    sigFlyTo = Signal(float, float, float)

    @Slot(str)
    def receive_from_js(self, msg):
        print(f"JS로부터 수신: {msg}")

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("PySide6 + CesiumJS 연동")
        self.resize(1024, 768)

        # 위젯 구성 (Qt Designer를 썼다면 setupUi(self)로 대체)
        central_widget = QWidget(self)
        self.setCentralWidget(central_widget)
        layout = QVBoxLayout(central_widget)

        self.web_view = QWebEngineView()
        self.btn_move = QPushButton("서울 상공으로 이동 (Python -> Cesium)")
        layout.addWidget(self.web_view)
        layout.addWidget(self.btn_move)

        # WebChannel 연결

```

```

self.channel = QWebChannel()
self.bridge = Bridge()
self.channel.registerObject("pyBridge", self.bridge)
self.web_view.page().setWebChannel(self.channel)

# HTML 로드
html_path = os.path.abspath("cesium_view.html")
self.web_view.setUrl(QUrl.fromLocalFile(html_path))

# 버튼 클릭 시 시그널 방출 -> JS의 camera.flyTo 실행
self.btn_move.clicked.connect(lambda: self.bridge.sigFlyTo.emit(37.5665, 126.

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

```

4. 실무 구현 시 고려사항

- **오프라인/보안 폐쇄망 환경:** CDN 링크 대신 CesiumJS 빌드 파일(`Cesium.js` , `widgets.css` , `Assets` 등)을 로컬 디렉터리에 다운로드하여 HTML 상대 경로로 참조해야 합니다.
- **대용량 동적 렌더링:** 수천 개의 궤적 또는 점 데이터를 실시간으로 보낼 때는 매번 개별 시그널을 쏘기보다 **CZML(Cesium Language)** 형태의 JSON 문자열 또는 CZML 스트림 파일로 변환하여 `viewer.dataSources.add(Cesium.CzmlDataSource.load(...))` 방식으로 밀어넣는 것이 렌더링 성능에 유리합니다.
- **C++ Native 연동 대안:** Python 브라우저 임베딩 외에 완전한 네이티브 데스크톱 C++ 앱 레벨에서는 **Cesium Native**(Unreal Engine/Unity/O-3DE 플러그인 기반 코어)를 연동하기도 하지만, 일반적인 Python 데스크톱 GUI에서는 `PySide6 + QWebEngine + CesiumJS` 조합이 가장 안정적이고 구현 난이도가 낮습니다.

우주감시레이다 체계 개발을 위한 종합 시뮬레이션 툴을 만들려고 한다면, 핵심 알고리즘 부터 개발하고 그 다음에 GUI를 만드는 게 좋을까? 기본적인 GUI를 먼저 디자인하고 그 다음에 핵심 알고리즘을 만들어서 붙히는 게 좋을까?

결론부터 말씀드리면, **핵심 알고리즘(백엔드 엔진)**을 먼저 개발하고 독립적으로 검증한 뒤, **GUI(프론트엔드)와 연동하는 방식(Backend-First / API-Driven)**이 압도적으로 유리합니다.

구상하신 우주감시레이다 종합 시뮬레이션 툴은 연산량이 극도로 많은 복잡한 시스템입니다. 이 규모의 체계를 개발할 때 GUI부터 설계하거나 GUI 코드와 연산 코드를 섞어서 짜게 되면 치명적인 구조적 한계에 부딪히게 됩니다.

그 이유와 가장 효율적인 개발 순서를 제안해 드립니다.

1. 왜 핵심 알고리즘부터 개발해야 하는가?

- **연산 루프와 렌더링 루프의 분리 필수:** 수만 개의 위성을 1초 이하 단위로 SGP4/HPOP 전파하고, AESA 레이더의 다중 표적 추적(칼만 필터)을 계산하는 루프를 GUI(예: PyQt, PySide6) 메인 스레드에 올리면 UI 화면이 완전히 멈춰버리는(프리징) 현상이 발생합니다. 파이썬은 계산만 전담하는 백엔드 서버로 작동시키고, GUI와 3D 렌더러(예: Cesium.js)는 독립된 프로세스로 분리해야 합니다.
- **자료 구조의 최적화:** 대규모 위성 카탈로그를 처리하려면 객체지향적인 `class` 기반 설계 대신, CPU 캐시 히트율과 Numba/GPU 가속을 극대화할 수 있는 NumPy/PyTorch 기반의 구조체 배열(SoA, Structure of Arrays) 형태로 알고리즘이 먼저 설계되어야 합니다. GUI에 맞춰 데이터를 짜면 이 고속화 구조를 놓치기 쉽습니다.
- **모듈 독립성 및 테스트 용이성:** 구상하신 5번 항목("각각의 모듈이 독립적으로 시뮬레이션 해볼 수 있는")을 만족하려면, GUI가 없더라도 파이썬 스크립트나 커맨드라인(CLI)만으로 SGP4 전파, 레이더 빔 제어, 궤도 결정 등의 단위 모듈이 완벽히 동작해야 합니다.

2. 권장하는 단계별 개발 로드맵

[1단계] 코어 엔진 및 자료구조 설계 (Core Backend)

가장 먼저 우주 공간의 물리 엔진과 데이터 파이프라인을 구축합니다.

- **통합 데이터 구조 정의:** 수만 개의 우주 물체 상태 벡터를 담을 NumPy 구조화 배열이나 PyTorch 텐서를 정의합니다.
- **모듈 1(우주물체 모의):** 좌표계 변환(ECI/TEME/ITRF), SGP4 모듈을 Numba(`@njit`, `prange`)로 가속화하여 구현합니다.
- **모듈 2 & 3 (레이더 모의 및 후처리):** AESA 레이더 관측치 생성과 EKF/UKF 기반 궤도 결정 알고리즘을 텐서 연산으로 구현하고, 근접 충돌 확률(CA) 계산을 위해 AP/APP/APS 필터 등의 알고리즘을 구현하여 검증합니다.

[2단계] 데이터베이스 및 통신 미들웨어 구축 (Data Layer)

수많은 모듈이 데이터를 주고받고 저장할 수 있는 뼈대를 만듭니다.

- 실시간 탐지 데이터나 현재 위성 상태는 Redis와 Kafka 같은 인메모리/메시지 큐로 처리합니다.

- 과거 궤도 데이터, 레이더 관측치, 공분산 데이터는 시계열 최적화 DB인 TimescaleDB(PostgreSQL)에 적재하는 파이프라인을 구성합니다.
- 이 백엔드 엔진이 GUI와 통신할 수 있도록 데이터를 무거운 JSON 대신 바이너리 데이터(ArrayBuffer 등)로 직렬화하여 쏘아주는 통신 채널(WebSocket 등)을 만듭니다.

[3단계] GUI 및 3D 시각화 통합 (Frontend Integration)

이제 엔진은 완성되었으므로, 사용자가 이를 쉽게 다룰 수 있는 조종석(GUI)을 만듭니다.

- **UI 레이아웃 디자인:** PySide6 + Qt Designer를 이용해 모듈별 탭(우주물체 조회, 레이더 제어, CA 분석, DB 검색 등)을 구성합니다.
- **3D 렌더링 통합:** 앞서 질문하신 **CesiumJS**를 PySide6의 `QWebEngineView`에 임베딩합니다. Python 백엔드에서 1단계에 구현한 수만 개의 위성 좌표를 바이너리로 WebEngine에 넘겨주어, GPU 인스턴싱으로 3D 지구본 위에 부드럽게 표출합니다.
- **그래프 및 표 연동:** Matplotlib, PyQtGraph 등을 활용해 레이더 탐지 이력, 궤도 요소 변화, 충돌 확률 추이 등을 차트로 표출하고 DB 검색 결과를 테이블에 바인딩합니다.

요약

"핵심 알고리즘(단위 기능 구현) → 고속 통신 및 DB 아키텍처 → GUI 설계 및 연동" 순서로 진행하시는 것이 정석입니다.

초기에는 GUI 화면 대신 주피터 노트북이나 파이썬 콘솔에서 1) TLE 로드 → 2) SGP4 전파 → 3) 결과값 검증을 수행하며 **알고리즘의 신뢰성과 연산 속도**를 튜닝하는 데 집중하시기 바랍니다. 엔진이 튼튼하고 통신 인터페이스가 명확히 분리되어 있으면, 나중에 GUI를 붙이거나 웹 기반 서비스로 전환하는 것은 매우 쉽습니다.